

先治理再连接：生产级知识图谱里的身份与本体打标

Curate Before You Connect

本篇范围说明：源文档是一篇 29 页的工程实践论文 (arXiv:2608.10644)。下面「全文翻译」按论文章节推进，覆盖摘要、引言、身份阶梯、那次身份事故、本体打标的证据不对称、静默正确性故障、治理债务、人工闸门与队列、以及分诊 agent 的写入门槛；**不含参考文献列表、公式的完整符号定义与附录表格。**

核心内容

Q：这篇论文讲的是哪一段工作？ A：抽取之后、写进图之前那一段。前一篇姊妹论文讲怎么把杂乱的文档流变成经校验的实体与关系；这篇从那里接手，讲把抽取结果写进一张会被查询的图。作者说这一步通常被当作管道活 (plumbing)，而他们发现**关键决策全在这一步**。规模是：98,795 份政府文件，产出 537,157 个实体、2,198,567 条关系。

Q：为什么这一步的决策比抽取更关键？ A：因为**两类错误的可恢复性不对称**。抽取错了可以恢复——类型标错了可以改回来，关系漏抽了下一遍可以补。**身份错了不能恢复**：两条记录一旦被合并到同一个节点、属性也合在一起，把它们分开所需的信息就已经被销毁了。更糟的是这个操作是**静默的**：没有约束被违反，没有异常被抛出，产出的节点格式完美——它只是在描述一个不存在的东西。作者的判断是：这种形状的故障在被人发现之前传得最远，因为下游每一步都把它当数据用。

Q：这个不对称推出了什么设计规则？ A：三条，作者说整个设计都由它决定：**决策是机械的就毫不犹豫地自动化**；**需要判断的就上报给人**；而一个操作既是破坏性的、又难以察觉的，那就在任何置信阈值下都拒绝自动化。第三条是一次事故教的。

Q：那次事故具体是什么？ A：一个电网板块出现在两份文件里，一次名字带空格、一次同一个名字带下划线。相似度匹配器判它们是同一个实体（**按任何字符串度量都判得对**）并合并了。合并之后那个存活下来的节点得到了：属于另一个设施的坐标、一个 95,000 米的覆盖半径、一个错的实体类型（变成了 Location）、以及一个标记它是合成数据的 `is_illustrative` 标志。**合并之前那个节点是正确的。**

Q：更严重的连带损失是怎么发生的？ A：这次合并还删掉了另一份无关文件里的八个实体。机制值得摊开讲：这个系统的溯源是按文件记的——一个节点记着它是从哪些文件推导出来的，而「移除某份文件的贡献」那条维护路径靠走这份溯源来工作。合并把两条记录的溯源也合了，于是存活节点声称两份文件都是它的来源。后来一次**完全正确的**按文件操作，依据这个已经错了的声称，把经由被吸收的溯源可达的实体当成了正在处理那份文件的东西，删掉了。**属性合并本身没删任何东西，它污染了一个后来那条正确例程所依赖的账本。**

Q：本体打标那一节最有价值的发现是什么？ A：名字不是类型断言。打标器有两种证据，作者起初把它们当等价的：一个是类型字符串，那是断言——有人（模型、schema、人）声称这个实体属于那个类；而一个实体的名字什么都没断言，它只是贴在一个个体上的标签，类型词恰好出现在里面这件事不构成任何声称。这就是类与实例的区别，混淆两者是一种已知的、能造出「名不符实的分类体系」的做法。

Q：这个混淆造成了什么具体后果？ A：作者的例子极清楚。「Government Resolution No. 30/2013」自己宣告了自己的类型，从里面取出 Government Resolution 是对的。而「Under Secretary (Regulations)」宣告的是一个职位，括号里限定的是哪个职位；从里面抽出 Regulations 去匹配类索引，得到 Statutory Rule 与 Legislation，于是一个人被分类成了法律条文。而变体生成器按自己的规格说明干得完全正确——括号规则的存在是为了把「Minehunter (MHC)」变成缩写 MHC，那是个真实有用的场合，它只是没有办法区分缩写和限定语。

Q：怎么修的？ A：改成只接受「类标签在名字里被锚定」的候选，而不是「仅仅出现在名字里」。形式化的判据是：类标签要么开在名字的最前面（前缀关系），要么它本身至少两个词元、且作为连续子序列出现在名字里。这让名字证据成为一个比断言型类型更弱的信号，而不是等权的信号。效果：在一份富化样本上，角色赋值从 36 条降到 4 条，且这 4 条全部经确认正确。

背景补充 / 点评

这是一篇工程实践论文，作者来自 Konectu，处理的是政府文件构成的生产级知识图谱。它不是方法论文——没有新算法、没有基准刷分，价值全在「我们踩了什么坑、代价是什么、后来改成什么」。

几处需要补的背景：

- **溯源（provenance）** 按文件记是知识图谱工程里常见的做法：每个节点记住自己来自哪些源文件，这样撤回一份文件的影响时可以只删它贡献的部分。这个做法本身没问题，出事的是「合并两个节点时也把溯源合了」——一份错的账本，让一条正确的清理例程去删了不该删的东西。
- **实体消解（entity resolution）** 是判断「两条记录是不是同一个东西」的那类技术，字符串相似度是最常见的实现。作者的转向不是调阈值，而是**改政策**：消解照跑、照算相似度、照排候选，但**不再合并任何东西**，候选进入人工队列由人决定。
- **一致性债务（conformance debt）** 指图里的数据与本体声明之间的偏差累积量。作者专门量化了这个数字，并给出一个反直觉的观察：有时次级分类比主分类更正确，因为主分类挂错了父类。

这篇论文最难得的地方是它对自己的诚实，而且诚实到不利于自己。三处：

第一，摘要里就写「我们是论证而非证明」("We argue rather than demonstrate that this is where the automation line belongs")，并且明说：没有报身份基准，而且这套键所允许的过度合并按构造就是检测不到的。一篇讲身份决策的论文承认自己没有身份基准，这需要克制。

第二，「一个没人处理的队列是一份记录，不是一条工作流。」他们的实体消解候选队列里有 3,304 对候选（1,002 高置信、2,302 中置信），一对都没被裁决过。作者自己把这句话摆出来：在有人真的开始处理之前，打标签只是推迟了一个决定，不是做出了一个决定。

第三，**把治理队列的算术摆出来**：人工累计接受 722 条、拒绝 53 条，合计 775 个决定；而队列里积压 48,403 条（第一次看的时候是 5,900 条）。流入远大于评审，而且每处理一份文件差距就再拉大一点。作者说这不是新观察——**生物信息学的人工治理二十年前就撞上过同一道算术**。

留白的地方。那个「锚定」判据（类标签要么是前缀、要么是至少两词元的连续子序列）在他们的样本上把 36 条降到 4 条，效果显著，但**论文没有报召回**——降下去的那 32 条里有多少是真该保留的？「4 条全部正确」只说了精确率。这一点作者在别处很克制，唯独这里没提。

对我自己的启示：最该拿走的四条

一、他们那段对「静默故障」的描述，是我一直在打的那类东西最准确的表述。

没有约束被违反，没有异常被抛出，产出的节点格式完美——它只是在描述一个不存在的东西。这种形状的故障在被人发现之前传得最远，因为下游每一步都把它当数据用。

我这两天处理的几件事全是这个形状：口径标签写错而数字一分不差（数字对账查不出来）；编号判错类别而答出一个正确的数字（对账只验数字有证据，不验证据说的是不是你问的那个东西）；渲染出一个页数正确、内容全空的 PDF 而退出码是 0。**以后描述这类问题，我用他们这个说法**。

二、那条三分法我要直接采用，尤其是第三条。

机械的决策毫不犹豫地自动化；需要判断的上报给人；**既是破坏性的、又难以察觉的，在任何置信阈值下都拒绝自动化**。

第三条正是我前两天做的那件事的理论依据——编号判不出归属就拒答，而不是去各个目录查一遍猜一个。当时我的论证是「按存在性猜仍然是猜」，他们给的论证更强：**这个操作的错误既不可逆、又检测不到，所以不该有一个置信阈值能让它通过**。

三、「名字不是类型断言」这句话，把我删掉那九处定制的理由讲清楚了。

我那九处删掉的东西，本质上都是「拿名字里出现的词去推断这个东西是什么」——按数字旁边的词判它属于哪个概念、按过滤器名字里的词判它断言了什么语义。他们给出了准确的说法：**类型字符串是断言，名字只是贴在个体上的标签**，类型词恰好出现在名字里不构成任何声称，混淆两者会造出一个名不符实的分类体系。

而他们那个「Under Secretary (Regulations)」变成法律条文的例子，比我举过的任何例子都锋利。更要紧的是那句：**变体生成器按自己的规格说明干得完全正确，它只是没有办法区分缩写和限定语**。这正是我反复说的「机制没错，是判据落错了地方」。

四、§ 3.6 那个批处理偏移量的坑，我要照它的规则改自己的判据。

打标器按批遍历「未打标」的实体，偏移量按批大小推进。而**打标会把行从选择器里移除**，所以一批成功打标 m 行就让总体缩小 m ，偏移量推进整个 b 就跳过了 m 行从未被检查的数据。长跑下来累积成：**大约一半总**

体没被碰过，而任务日志一路报着稳定进展、最后正常退出报成功。

他们的结论我直接抄下来：

完成度是从「尝试了多少工作」推断的，而不是对着「总体」量出来的。我们现在把「剩余数量是否真的到了零」当作这张图上任何批处理任务唯一可接受的终止条件。

这条我有直接对应的地方——我的评测、抓取、批量校验里都有「跑完了就算过了」的形状。**判据该是「剩余数是否到零」，不是「循环有没有跑完」。**

另外一条警告，是给我那个反问机制的。他们说「一个没人处理的队列是一份记录，不是一条 workflow」，3,304 对候选一对都没裁决。我的系统在判不清时拒答并把候选问回去——**如果没有人回答那次反问，拒答就只是把决定推迟了，不是做出了决定。**我目前没有任何机制观测「反问之后用户有没有回、回了之后答对了没有」。这个缺口和他们那个未处理队列是同一个。

文中金句

"A wrong type can be corrected later, but two records merged under one identity cannot be separated once their properties have been combined, and the merge leaves no error behind." 类型标错了以后可以改，但两条记录一旦以同一个身份合并、属性也合在一起，就再也分不开了——而这次合并不留下任何错误痕迹。

"No constraint is violated, no exception is raised, and the resulting node is perfectly well-formed. It is simply describing something that does not exist." 没有约束被违反，没有异常被抛出，产出的节点格式完美。它只是在描述一个不存在的东西。

"Faults of this shape are the ones that travel furthest before anyone notices, because every stage downstream treats them as data." 这种形状的故障在被人察觉之前传得最远，因为下游每一步都把它当数据用。

"Where a decision is mechanical we automate it without hesitation; where it requires judgement we escalate it; and where an operation is both destructive and hard to detect we decline to automate it at any confidence threshold." 决策是机械的，我们毫不犹豫地自动化；需要判断的，我们上报；而一个操作既是破坏性的、又难以察觉的，我们在任何置信阈值下都拒绝自动化。

"An entity's name is not an assertion of anything. It is a label attached to an individual, and the fact that a type word happens to appear inside it carries no claim at all." 一个实体的名字什么都没有断言。它只是贴在一个个体上的标签，而类型词恰好出现在里面这件事，不构成任何声称。

"An unworked queue is a record, not a workflow." 一个没人处理的队列是一份记录，不是一条工作流。

"Every individual operation succeeded, every log line was accurate, and the aggregate claim was false, because completion was inferred from work attempted rather than measured against the population." 每一个单独的操作都成功了，每一行日志都准确，而整体的那个结论是假的——因为完成度是从「尝试了多少工作」推断出来的，而不是对着总体量出来的。

全文翻译

摘要

Extraction produces candidate entities and relationships; writing them into a graph is where identity is decided, and identity decisions are destructive in a way extraction errors are not. A wrong type can be corrected later, but two records merged under one identity cannot be separated once their properties have been combined, and the merge leaves no error behind.

抽取产出候选的实体与关系；而把它们写进图的那一步，才是身份被决定的地方，而身份决策的破坏性是抽取错误所没有的。类型标错了以后可以改，但两条记录一旦以同一个身份合并、属性也合在一起，就再也分不开了——而这次合并不留任何错误痕迹。

This paper describes the ingestion and ontology-tagging layer that turns a validated extraction stream into a knowledge graph of 537,157 entities and 2,198,567 relationships drawn from 98,795 government documents.

本文描述的是那一层「灌入与本体打标」的工作：它把一条经校验的抽取流，变成一张来自 98,795 份政府文件、含 537,157 个实体与 2,198,567 条关系的知识图谱。

We describe a record-identity ladder that decides sameness from identifier columns, name columns, display names and type-scoped position rather than from name similarity. The ladder governs de-duplication within parsed tables, while the graph write applies a coarser canonical-name key, so records sharing a canonical name merge automatically on exact equality.

我们描述一个「记录身份阶梯」，它判断「是否同一个」的依据是标识符列、名称列、显示名与按类型限定的位置，**而不是名字相似度**。这个阶梯管的是解析出的表格内部的去重；而写入图时用的是一个更粗的规范名

键，所以共享同一个规范名的记录在精确相等时自动合并。

We argue rather than demonstrate that this is where the automation line belongs: no identity benchmark is reported, and the over-merges the key permits are undetectable by construction.

我们是在论证而不是在证明「自动化的界线该划在这里」：本文没有报告身份基准，而且这套键所允许的过度合并，按构造就是检测不到的。

That policy, under which entity resolution only ever flags candidates, followed an incident in which two surface forms of one name were merged, corrupting a correct record and deleting eight entities from an unrelated document.

那条「实体消解只标记候选、从不合并」的政策，是在一次事故之后定的：同一个名字的两表面形式被合并，污染了一条正确的记录，并且删掉了另一份无关文件里的八个实体。

We then describe multi-class ontology tagging and an evidence asymmetry we did not anticipate: an entity name is an instance label rather than a type assertion, so matching name fragments against a class index invents classifications. Requiring anchored evidence cut role assignments on an enriched sample from 36 to 4, all confirmed correct.

接着我们描述多类本体打标，以及一个我们没有预料到的证据不对称：**实体的名字是一个实例标签，而不是一个类型断言**，所以拿名字的片段去匹配类索引，等于凭空造出分类。改成要求证据被锚定之后，在一份富化样本上，角色赋值从 36 条降到 4 条，且这 4 条全部经确认正确。

We quantify the graph's conformance debt, show secondary classifications compensating for a mis-parented primary class, and describe a curation queue grown to 48,403 pending proposals against 775 human decisions.

我们量化了这张图的一致性债务，展示了次级分类如何补偿一个挂错父类的主分类，并描述了一个治理队列——它已经积压到 48,403 条待审提案，而人工累计只做了 775 个决定。

一、引言：这一步不是管道活

A companion paper describes the extraction layer that converts a heterogeneous document stream into validated entities and relationships. This paper begins where that one ends: with extraction output in hand, and the problem of writing it into a graph that will be queried.

一篇姊妹论文描述了抽取层——把一条异质的文档流转换成经校验的实体与关系。本文从那一篇结束的地方开始：抽取输出已经在手，问题是怎么把它写进一张会被查询的图。

That step is usually treated as plumbing. We found it is where the consequential decisions are made. Extraction errors are recoverable: a mistyped entity can be retyped, a missed relationship can be re-extracted on a later pass. Identity errors are not.

这一步通常被当作管道活。我们发现它才是那些有后果的决策发生的地方。抽取错误是可恢复的：类型标错的实体可以重新标，漏抽的关系可以在后一遍补上。**身份错误不可恢复。**

Once two records have been merged under a single node and their properties combined, the information required to separate them has been destroyed. Worse, the operation is silent: no constraint is violated, no exception is raised, and the resulting node is perfectly well-formed. It is simply describing something that does not exist. Faults of this shape are the ones that travel furthest before anyone notices, because every stage downstream treats them as data.

一旦两条记录被合并到同一个节点、属性也合在一起，把它们分开所需的信息就已经被销毁了。更糟的是这个操作是静默的：没有约束被违反，没有异常被抛出，产出的节点格式完美。它只是在描述一个不存在的东西。这种形状的故障在被人察觉之前传得最远，因为下游每一步都把它当数据用。

This asymmetry drives the whole design. Where a decision is mechanical we automate it without hesitation; where it requires judgement we escalate it; and where an operation is both destructive and hard to detect we decline to automate it at any confidence threshold.

这个不对称决定了整个设计。决策是机械的，我们毫不犹豫地自动化；需要判断的，我们上报；而一个操作既是破坏性的、又难以察觉的，我们在任何置信阈值下都拒绝自动化。

二、那次身份事故，以及它的代价

The ladder replaced a similarity-based matcher, and it did so because that matcher caused a production incident that we describe here in full, because the detail is the argument.

这个阶梯取代的是一个基于相似度的匹配器，而取代的原因是那个匹配器引发了一次生产事故。我们在这里完整描述它，**因为细节本身就是论证。**

A power-grid sector appeared in two documents, once with a spaced name and once with the same name underscored. A similarity matcher scored these as the same entity (correctly, by any string metric) and merged them. The merge combined properties from both records.

一个电网板块出现在两份文件里，一次名字里带空格，一次同一个名字带下划线。相似度匹配器把这两个判成同一个实体（**按任何字符串度量来说，判得都对**）并合并了它们。合并把两条记录的属性合到了一起。

The surviving node acquired coordinates belonging to a different facility, a coverage radius of 95,000 metres, an entity type of Location in place of its correct type, and an `is_illustrative` flag marking it as synthetic. Before the merge the node had been correct.

存活下来的那个节点获得了：属于另一个设施的坐标、一个 95,000 米的覆盖半径、一个 Location 类型（取代了它原本正确的类型）、以及一个把它标记为合成数据的 `is_illustrative` 标志。**在合并之前，那个节点是正确的。**

The merge additionally deleted eight entities belonging to an unrelated document. The mechanism is worth spelling out, because it is the part that turns a bad classification into data loss. Provenance in this system is per-document: a node records the files it was derived from, and the maintenance path that removes a document's contribution works by walking that provenance.

这次合并还额外删掉了另一份无关文件里的八个实体。这个机制值得摊开讲，因为**正是它把一次错误的分类变成了数据丢失**。这个系统里的溯源是按文件记的：一个节点记着它是从哪些文件推导出来的，而那条「移除某份文件的贡献」的维护路径，靠走这份溯源来工作。

When the merge combined the two records it also combined their provenance, so the surviving node claimed both documents as sources. A subsequent per-document operation, acting on that now-incorrect claim, treated entities reachable through the absorbed provenance as belonging to the document being processed, and removed them.

当合并把两条记录合起来时，它也把两者的溯源合了，于是存活的节点声称两份文件都是它的来源。后来一次按文件进行的操作，依据这个此时已经错了的声称，把「经由被吸收的溯源可达的实体」当成了正在处理的那份文件的东西，并把它们移除了。

The property merge did not delete anything by itself. It corrupted the bookkeeping that a later, entirely correct routine depended on: which is why the damage surfaced somewhere other than where it was caused.

属性合并本身没有删掉任何东西。它污染的是一个后来那条**完全正确的**例程所依赖的账本——**这就是为什么损害显现的地方，不是它被造成的地方。**

三、名字与类型的证据不对称

The tagger has two sources of evidence, and we initially treated them as interchangeable. They are not, and the difference is the most useful thing we learned building this layer.

打标器有两个证据来源，我们起初把它们当成可以互换的。它们不是，**而这个区别是我们建这一层时学到的最有用的东西。**

A type string is a type assertion. Something (a model, a schema, a human) has claimed that this entity is of that kind. An entity's name is not an assertion of anything. It is a label attached to an individual, and the fact that a type word happens to appear inside it carries no claim at all. The distinction is the class/instance one, and conflating the two is a known way to build a taxonomy that does not mean what it says.

一个类型字符串是一次类型断言：某个东西（一个模型、一份 schema、一个人）声称了这个实体属于那一类。而一个实体的名字什么都没有断言。它只是贴在一个个体上的标签，而类型词恰好出现在里面这件事，不

构成任何声称。这个区别就是类与实例的区别，[而混淆两者是一种已知的、能造出名不符实的分类体系的做法。](#)

The distinction is invisible until you look at what it produces. "Government Resolution No. 30/2013" announces its own type; taking Government Resolution from it is right, in the way a lexico-syntactic pattern is right. "Under Secretary (Regulations)" announces an office, and the parenthetical qualifies which office. Extracting Regulations from it and matching that against the class index yields Statutory Rule and Legislation, so a person ends up classified as legislation.

这个区别在你看它产出什么之前是看不见的。「Government Resolution No. 30/2013」宣告了自己的类型，从里面取出 Government Resolution 是对的，就像一个词法句法模式是对的那样。「Under Secretary (Regulations)」宣告的是一个职位，而括号里限定的是哪一个职位。从里面抽出 Regulations 去匹配类索引，得到的是 Statutory Rule 与 Legislation，[于是一个人被最后被分类成了法律条文。](#)

Our variant generator was doing exactly this, and doing it correctly by its own specification: the parenthetical rule exists to turn "Minehunter (MHC)" into the abbreviation MHC, which is a real and useful case. It simply has no way to tell an abbreviation from a qualifier.

我们的变体生成器干的正是这件事，而且[按它自己的规格说明干得完全正确](#)：括号规则的存在是为了把「Minehunter (MHC)」变成缩写 MHC，那是个真实且有用的场合。[它只是没有办法区分一个缩写和一个限定语。](#)

We now admit a name-derived candidate only when the class label is anchored in the name rather than merely present in it, which makes name evidence a weaker signal than an asserted type rather than an equal one. In words: the class label either opens the name, or appears inside it as a phrase of at least two tokens. Requiring anchored evidence cut role assignments on an enriched sample from 36 to 4, all confirmed correct.

我们现在只接受这样的名字派生候选：类标签[在名字里被锚定](#)，而不是仅仅出现在名字里。这让名字证据成为一个比「被断言的类型」更弱的信号，而不是与之等权的信号。用话说就是：类标签要么开在名字的最前面，要么作为一个至少两个词元的短语出现在名字内部。要求证据被锚定之后，在一份富化样本上，角色赋值从 36 条降到 4 条，且这 4 条全部经确认正确。

四、打标器里的一次静默正确性故障

The tagger walks the untagged population in batches, and the obvious way to write that walk is with an offset that advances by the batch size. That is wrong here, and it was wrong in our code for some time.

打标器按批遍历「尚未打标」的总体，而写这个遍历最顺手的方式是让偏移量按批大小推进。[在这里那是错的](#)，而且在我们的代码里错了一段时间。

The selector is untagged entities. Tagging a row removes it from the selector. So a batch that successfully tags m of its b rows shrinks the population by m , and an offset advanced by the full b skips m rows that were never examined. Over a long run this compounds: roughly half the population went untouched while the job logged steady progress and exited reporting success.

选择器选的是「未打标的实体」。给一行打上标，就把它从选择器里移除了。所以一批里成功打标 m 行，就让总体缩小 m ，而偏移量推进整个 b ，就跳过了 m 行**从未被检查过的数据**。长跑下来这会累积：**大约一半的总体没被碰过，而任务日志一路记录着稳定的进展，最后正常退出、报告成功。**

This is the same defect as the backfill wearing different clothes. In both cases every individual operation succeeded, every log line was accurate, and the aggregate claim was false, because completion was inferred from work attempted rather than measured against the population. We have started treating "did the count of remaining items actually reach zero" as the only acceptable termination condition for any batch job over this graph.

这和之前那次回填是同一个缺陷换了身衣服。两次都是：每一个单独的操作都成功了，每一行日志都准确，而整体的那个结论是假的——**因为完成度是从「尝试了多少工作」推断出来的，而不是对着总体量出来的。**我们现在把「剩余数量是否真的到了零」当作这张图上任何批处理任务**唯一可接受的终止条件**。

五、人工闸门：只标记不合并

After the incident we changed the policy rather than the threshold. Entity resolution still runs, still computes similarity, and still ranks candidate pairs. It no longer merges anything. Candidates are written to an administrative queue and a person decides.

那次事故之后，我们改的是**政策**，不是阈值。实体消解照跑，照算相似度，照给候选对排序。**它不再合并任何东西**。候选被写进一个管理队列，由人来决定。

At the point of measurement that queue held 3,304 candidate pairs (1,002 ranked high-confidence and 2,302 medium) none of them applied, and none of them adjudicated either. We should say plainly what that means: an unworked queue is a record, not a workflow. This one has never been worked, and until it is, flagging defers a decision rather than making one.

在测量的那个时点，那个队列里有 3,304 对候选（1,002 对排为高置信，2,302 对为中等），**没有一对被应用，也没有一对被裁决**。我们应当把这意味着什么说明白：一个没人处理的队列是一份记录，不是一条**工作流**。这一个从来没被处理过，而在它被处理之前，打标记只是推迟了一个决定，而不是做出了一个决定。

六、治理队列的算术

Extraction does not only produce entities. When it encounters a concept the ontology cannot express, it proposes one: a candidate term with its evidence, the documents it appeared in, a suggested parent, and the domain and range observed in text. Proposals are not applied. They wait in a review queue.

抽取不只产出实体。当它遇到一个本体表达不了的概念时，它会提一个：一个候选术语，连着它的证据、它出现过的文件、一个建议的父类、以及在文本里观察到的定义域与值域。**提案不会被直接应用**，它们在一个评审队列里等着。

The mechanism works. What stopped working is the arithmetic around it. Humans have accepted 722 proposals and rejected 53. The queue currently holds 48,403 pending items against roughly 5,900 when we first looked at it. Inflow exceeds review by a wide and growing margin and the gap widens with every document processed.

这个机制是有效的，失效的是它周围的算术。 人工累计接受了 722 条提案、拒绝了 53 条。队列当前积压 48,403 条，而我们第一次看它的时候是大约 5,900 条。流入远大于评审，差距还在扩大，每处理一份文件就再拉大一点。

That manual curation does not scale with automated extraction is not a new observation; it is the same arithmetic that biocuration ran into two decades ago.

「人工治理跟不上自动化抽取」不是一个新观察——**这就是生物信息学的人工治理二十年前撞上的同一道算术。**

七、分诊 agent: 怎么挣到写入的权利

The agent runs read-only by default. Producing a report is the normal mode; writing requires an explicit flag, and the commit path asserts it rather than trusting the caller.

这个 agent 默认只读。产出一份报告是常态模式；写入需要一个显式的开关，**而提交那条路径自己去断言这个开关，不信任调用方。**

Before writes are enabled at all, a run's verdicts are sampled and checked by a person, and we require agreement on at least 28 of 30 sampled decisions with no incorrect acceptances. That fraction is a point estimate near 93% with a lower 95% bound around 78%; for a component writing to a shared client deliverable the bound is the number that matters, and thirty samples is a weak gate we would widen before trusting it broadly.

在允许写入之前，一次运行的判定会被抽样、由人复核，我们要求 30 个抽样决定里至少 28 个一致，且**没有任何错误的「接受」**。这个比例是一个约 93% 的点估计，95% 置信下界大约 78%；**对一个要写进共享交付物的组件来说，起作用的是那个下界**，而三十个样本是一道弱闸门，在广泛信任它之前我们会把它加宽。

Terms the agent creates are marked with its identity and a run identifier, so a run can be reversed as a unit. Every write is idempotent, so re-running a batch changes nothing, and every proposal update is guarded on the proposal still being pending, so a human editing the same item concurrently always wins.

这个 agent 创建的术语会被打上它自己的身份与一个运行标识，**所以一次运行可以作为一个整体被撤销**。每一次写入都是幂等的，所以重跑一批什么也不改变；而每一次提案更新都以「该提案仍处于待审状态」为前置

守卫，所以同时在编辑同一条的人永远赢。

参考链接

- [arXiv:2608.10644](#) — 本文的 arXiv 页面 (Curate Before You Connect, 2026 年 8 月 11 日)
- [SHACL \(W3C\)](#) — 与本文「一致性债务」相关的图校验标准
- [SKOS \(W3C\)](#) — 分类体系建模标准，本文讨论的类／实例混淆问题在这里也是经典议题